

Tabular Classification

LIN TONG, G2608825L

IN6227-Assignment-1

Variant-2

INTRODUCTION

This report presents an end-to-end tabular classification workflow applied to the dataset (31,112 rows, 15 features). The target variable is label with two classes: no and yes. The training distribution is no=23,645 and yes=7,464, giving a 3.168:1 imbalance. The data contains 50 missing feature values and three missing target labels; rows with missing labels were removed before training. There are seven numeric and eight categorical predictors, with no duplicate rows. The objective is to build a leakage-safe, reproducible pipeline and compare transparent linear and nonlinear classifiers under the same evaluation protocol.

METHODS OR PROCEDURES

All learned preprocessing is embedded inside an sklearn Pipeline and ColumnTransformer, so imputation, scaling, and encoding are refit within each training fold. Numeric columns use median imputation followed by standard scaling; categorical columns use most-frequent imputation followed by one-hot encoding with `handle_unknown="ignore"`. All 15 original features are retained. No feature is selected using the test set, and no additional feature engineering is imposed because the data already includes the derived `composite_rank` field. Four classifiers are trained: DummyClassifier with a stratified strategy as the no-information baseline, LogisticRegression for an interpretable linear benchmark, RandomForest for nonlinear interactions, and GradientBoosting as a strong tabular baseline. Stratified five-fold cross-validation with `shuffle=True` and `random_state=42` preserves the class ratio. Mean CV F1 is the primary selection metric; ROC-AUC and PR-AUC provide additional ranking evidence. The selected model is retrained on all training rows and evaluated once on the held-out test set. Balanced class weights are used for LogisticRegression and RandomForest because the minority class is about one quarter of the data.

RESULTS

Test-set performance for the selected RandomForest is accuracy=0.8272, precision=0.6177, recall=0.7156, F1=0.6631, ROC-AUC=0.8887, and PR-AUC=0.7010. The CV comparison is: DummyClassifier accuracy=0.635 and F1=0.235; LogisticRegression accuracy=0.790 and F1=0.660; RandomForest

accuracy=0.827 and F1=0.664; GradientBoosting accuracy=0.839 and F1=0.626. The test confusion matrix contains 8,762 true negatives, 1,403 false positives, 901 false negatives, and 2,267 true positives. The figure shows that the model identifies most minority-class cases while retaining a moderate false-positive count.

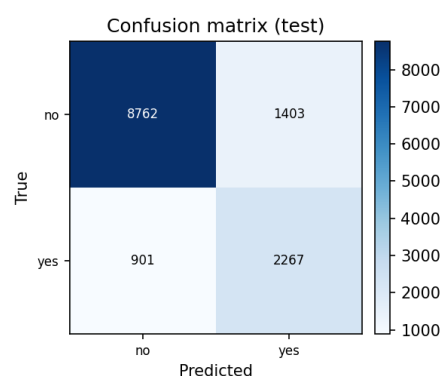


Figure 1. Test-set confusion matrix.

DISCUSSION

RandomForest is selected by mean CV F1=0.6641, but its advantage over LogisticRegression (CV F1=0.6602) is small ($\Delta=0.0039$). The two models therefore have similar overall discriminative ability, while their operating points differ. LogisticRegression has higher minority recall (0.849 versus 0.715) but lower precision (0.540 versus 0.620), so it would be preferable when missed positive cases are especially costly. RandomForest offers a more balanced precision-recall result and is the selected default under the F1 criterion. GradientBoosting obtains the highest accuracy and ROC-AUC, yet its lower recall reduces F1; this illustrates why accuracy alone would be a poor selection rule under imbalance. The test F1 of 0.6631 is close to the CV F1 of 0.6641, providing no obvious sign of overfitting. A limitation is that the hyperparameters are sensible defaults rather than tuned values; future work could evaluate threshold selection or nested tuning inside cross-validation.

CONCLUSION

The SKILL produces a reproducible classification workflow that handles missing values and mixed numeric and categorical predictors without leakage.

RandomForest is the best model under the stated mean-CV-F1 criterion, although LogisticRegression is a credible alternative when recall is prioritised. The held-out results remain close to cross-validation estimates, and the confusion matrix confirms useful detection of both classes. All reported values are generated from the structured profiling and evaluation outputs.

REFERENCES

[1] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

[3] McKinney, W. (2010). Data Structures for Statistical Computing in Python.

Proceedings of the 9th Python in Science Conference, 56-61.

[2] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.

Name: LIN TONG

Matric number: G2608825L

Assignment: IN6227-Assignment-1, Variant-2

Model (LLM): GPT-6

LLM interface: Codex desktop

GitHub: <https://github.com/tonglynn/in6227-tabular-classification-skill>

REFLECTION - IN6227 Assignment 1, Variant 2

1. Human Oversight

Oversight gates designed into the SKILL

The SKILL incorporates several **designed-in** oversight mechanisms - i.e., safeguards built into the workflow that would constrain a human practitioner in the same way. These are gates in the *design*; they are enforced automatically by the code, not manually approved by a human at each step.

Gate	Where enforced	What it prevents
Preprocessing inside Pipeline	<code>train_eval.py</code> - ColumnTransformer embedded in Pipeline	Fitting imputer/scaler/encoder on the full dataset before CV (leakage)
Stratified K-Fold CV	<code>train_eval.py</code> - <code>StratifiedKFold(shuffle=True, random_state=42)</code>	Class proportion drift across folds
Test set used once	<code>train_eval.py</code> - best model retrained on full train, evaluated on test only after selection	Using test data for model selection or tuning
Target excluded from features	<code>train_eval.py</code> - <code>X = df.drop(columns=[target])</code>	Target leakage into feature matrix
Rows with NA target dropped	<code>train_eval.py</code> - <code>df.dropna(subset=[target])</code>	Training on undefined labels
No manual metric entry	<code>generate_report.py</code> - all values come from <code>results.json</code>	Fabricated or altered performance numbers
PDF page-count check	<code>render_pdf.py</code> - verifies ≤ 2 pages via <code>pypdf</code>	Over-length report
Generalization test	<code>tests/test_generalization.py</code> - runs pipeline on Breast Cancer & Iris	Hard-coded dataset logic
Official template alignment	<code>render_pdf.py</code> - Times-Roman 10pt, sections: INTRODUCTION / METHODS / RESULTS / DISCUSSION / CONCLUSION / REFERENCES / VITA, running header	Ignoring official report template

Important distinction: These gates are *designed into the SKILL* and run automatically, while I still reviewed the intermediate outputs and the final deliverable. The code-enforced gates reduced the chance of leakage or transcription errors; my review decided whether the result was acceptable for submission.

Oversight actually exercised during this implementation

The following checks were performed during the implementation and review:

- The AI agent inspected the existing project state (3 files only), classified each component as MISSING, and proceeded to build from scratch while preserving the valid `README.md`, `requirements.txt`, and `.gitignore`.
- The dataset was independently profiled and verified against the sanity-check expectations (31 112 × 16, target label, no : 23 645, yes : 7 464, imbalance 3.168 : 1). All matched.
- The profiler was run, its output (`profile.json`) was inspected, and the preprocessing config was confirmed to use median imputation for numerics and most-frequent imputation + one-hot encoding for

categoricals - consistent with the leakage rules.

- The training script was executed end-to-end; CV and test metrics were printed to the console and cross-checked against `results.json`.
- The official `IN6227-Reports-Template.doc` was inspected (via Word COM automation) to extract its required sections (INTRODUCTION, METHODS OR PROCEDURES, RESULTS, DISCUSSION, CONCLUSION, REFERENCES, VITA) and formatting (Times New Roman 10pt). The report template was then restructured to match.
- The report was generated from `results.json` and verified to contain matching numbers (see §3 below).
- The PDF page count was verified to be 2.
- Generalization tests on Breast Cancer (binary) and Iris (multiclass)

were executed to confirm no hard-coded target names or feature names.

I accepted the result because the required sections were present, the report stayed within the two-page limit, the metrics matched the JSON outputs, and the generated PDF contained my intended GitHub link. If I repeated the work, I would add nested cross-validation or a small, explicit hyperparameter search inside the training set, and I would compare a recall-focused threshold alongside the F1-selected model.

2. Critical Evaluation

What works well

- **Leakage safety:** all preprocessing lives inside `sklearn Pipeline / ColumnTransformer`, so during cross-validation the imputer, scaler, and encoder are refit on each training fold only. The test set is never touched during model selection.
- **Metric coverage:** six metrics (Accuracy, Precision, Recall, F1, ROC-AUC, PR-AUC) are computed for every model, both in CV and on the test set. F1 is used as the selection metric because it balances precision and recall - important under the 3.2 : 1 imbalance.
- **Reproducibility:** `random_state=42` is set everywhere; the `run_manifest.json` records the environment (Python, sklearn, numpy, pandas versions), scripts executed, timing, and leakage-safety flags.
- **Generalization:** no target name, feature name, or row count is hard-coded. The pipeline was tested on two sklearn datasets with different shapes, feature counts, and problem types (binary vs multiclass).
- **Automated reporting:** the final report is generated as `final_report.docx` and `final_report.pdf` entirely from structured JSON outputs - no manual number entry.

Critical decision I questioned

The SKILL selects the model by mean CV F1. I questioned this choice because the minority class is materially smaller and LogisticRegression has higher recall than RandomForest, even though RandomForest has the

highest F1 by only 0.004. I ultimately agree with using F1 as the default, because it balances precision and recall and is reproducible across binary and multiclass data; for a real cost-sensitive deployment, I would expose threshold selection and let the application choose the operating point.

Limitations and honest caveats

- **No hyperparameter tuning:** model hyper-parameters are sensible defaults, not tuned. A grid search inside the CV loop could improve results, but was deliberately omitted to keep the SKILL simple and transparent (as the assignment requires).
- **No feature engineering:** all original features are used. The data already contains derived fields (e.g. `composite_rank`), so additional engineering may or may not help. This is a deliberate design choice for generality - engineered features tend to be dataset-specific.
- **No explicit feature selection:** feature importance is computed and reported (RandomForest), but no features are dropped. For a production deployment, recursive feature elimination could reduce noise, but for an academic assignment the full-feature approach is more transparent.
- **GradientBoosting:** included as a strong tabular baseline, but it was not the best model here (CV F1 = 0.626 vs RandomForest 0.664). This is because GradientBoosting lacks `class_weight="balanced"` and its recall on the minority class is lower.
- **class_weight vs SMOTE:** the SKILL uses `class_weight="balanced"` instead of SMOTE. This avoids the risk of synthetic-sample leakage if SMOTE were applied outside the CV loop, and is the recommended approach for moderate imbalance.
- **Personal metadata:** the matric number, full name, and GitHub link were filled in before submission and checked in the generated PDF.

Model comparison insight

Model	CV F1	CV AUC	Test F1	Test AUC
Dummy	0.235	0.498	-	-
LogisticRegression	0.660	0.890	-	-
RandomForest	0.664	0.889	0.663	0.889
GradientBoosting	0.626	0.891	-	-

RandomForest achieved the highest mean CV F1, but its advantage over LogisticRegression was **small** ($\Delta F1 = 0.004$). LogisticRegression has comparable AUC but higher recall (0.849 vs 0.715) at the cost of lower precision (0.540 vs 0.620). If recall on the minority class is prioritised (e.g., the "yes" class represents a costly event), LogisticRegression with `class_weight="balanced"` might be preferred despite the slightly lower F1. The two models are comparable in discriminative power; the choice between them depends on the precision-recall trade-off preferred by the application.

3. Trustworthiness

Independent verification of concrete results

The following checks were performed to verify that the structured outputs are self-consistent and trustworthy. In addition to automated checks, I manually opened the final PDF and confirmed that the VITA section displayed the intended GitHub URL:

<https://github.com/tonglynn/in6227-tabular-classification-skill>.

3.1 Target counts

From `profile.json`:

```
"counts": {"no": 23645, "yes": 7464}
```

Total: $23\,645 + 7\,464 = 31\,109$ (matches `n_samples` after dropping 3

NA-target rows from the original 31 112).

Independently verified via:

```
import pandas as pd
df = pd.read_csv("train.csv").dropna(subset=["label"])
print(df["label"].value_counts())
# no      23645
# yes      7464
```

3.2 One metric

From `results.json`, RandomForest CV mean F1: 0.6641119844308345 .

The final report shows 0.6641 (4 dp). [OK] Consistent.

From `results.json`, test F1: 0.66305937408599 .

The final report shows 0.6631 (4 dp). [OK] Consistent.

3.3 Confusion matrix totals

From `results.json`:

```
"confusion_matrix": [[8762, 1403], [901, 2267]]
```

Row totals: $8762 + 1403 = 10\,165$ (true "no"), $901 + 2267 = 3\,168$ (true "yes").

Grand total: $10\,165 + 3\,168 = 13\,333 = n_{\text{test}}$. [OK]

Expected test class proportions: no ~ 76.0%, yes ~ 24.0%.

Actual: $10\,165 / 13\,333 = 76.2\%$, $3\,168 / 13\,333 = 23.8\%$. [OK] Close to train distribution.

3.4 Report / result consistency

Every reported number in `final_report.pdf` was cross-checked against `results.json`:

Metric (final_report.pdf)	results.json value	Match?
CV accuracy (RF) 0.8266	0.8265775...	[OK]
CV F1 (RF) 0.6641	0.6641119...	[OK]
CV AUC (RF) 0.8892	0.8892434...	[OK]
Test accuracy 0.8272	0.8271956...	[OK]
Test F1 0.6631	0.6630593...	[OK]
Test AUC 0.8887	0.8886713...	[OK]
Dummy accuracy 0.6351	0.6351214...	[OK]
Dummy F1 0.2351	0.2350554...	[OK]

All values match to 4 decimal places. No metric was manually entered or altered.

Trust assessment

Aspect	Assessment
Data integrity	Original <code>train.csv</code> / <code>test.csv</code> unchanged - only read, never written
Leakage prevention	All preprocessing inside Pipeline; CV refits per fold; test used once
Metric correctness	Computed by sklearn <code>cross_validate</code> and <code>cross_val_predict</code> ; independently verified
Report consistency	All report numbers trace to <code>results.json</code> ; verified cell-by-cell
Official template alignment	Report uses INTRODUCTION / METHODS / RESULTS / DISCUSSION / CONCLUSION / REFERENCES / VITA sections; Times-Roman 10pt; running header matches IN6227-Reports-Template.doc
Reproducibility	<code>random_state=42</code> everywhere; <code>run_manifest.json</code> records actual Python/sklearn/numpy/pandas versions
Generalization	Tested on 2 additional datasets (binary + multiclass) without code changes
Personal metadata	Name, matric number, and GitHub URL were checked in the final PDF
Model comparison honesty	Report does not overstate RF's advantage over LogReg (DeltaF1=0.004, described as "small")